

Restaurant Discovery System

Design Documentation

Module: Oreo McFlurry | CSCI 275 | July 2026

Team Members

Student	Student ID
Borigas, Edrien Gappi	815023
Kaur, Rajvir	816815
Ngwira, Vitumbiko	817172
Singh, Gursharn Sangha	815612
Kaur, Ramanjeet	816695

1. Introduction

This document describes the design of the Restaurant Discovery module for the CSCI 275 group project. It covers the system architecture, tech stack, database design, API design, and UI/UX design decisions. The module enables users to search, filter, and interact with restaurant listings and save favorites.

1.1 Purpose

The purpose of this document is to provide a clear technical and design reference for the development team and course evaluator. It serves as the blueprint for how the Restaurant Discovery module is built, organized, and integrated with other modules.

1.2 Scope

The Restaurant Discovery module covers:

- Search and filtering of restaurant listings by name, cuisine, price, and rating
- Sorting of results by rating or newest
- Saving and removing restaurants to a personal favorites list
- Sharing restaurant links via clipboard

Out of scope: user authentication (handled by a separate module), restaurant owner management, table reservations, and social feeds.

2. System Architecture

The Restaurant Discovery module follows a three-tier architecture: a React frontend (client), a Node.js/Express backend (server), and a PostgreSQL database. Communication between tiers is done via HTTP requests and JSON responses.

2.1 Architecture Overview

Tier	Technology	Responsibility
Frontend	React (Vite)	User interface, sends HTTP requests to the backend
Backend	Node.js + Express	Receives requests, processes logic, queries database via Prisma
ORM	Prisma	Translates JavaScript into SQL queries for PostgreSQL
Database	PostgreSQL	Stores restaurant, user, and favorites data

2.2 Request Flow

The following describes the flow of a typical HTTP request through the system:

1. User interacts with the React frontend (e.g. types a search query)
2. React sends an HTTP GET request to the Express server (e.g. GET /restaurants?search=Italian)
3. Express receives the request and passes it to the relevant route handler
4. The route handler uses Prisma to query the PostgreSQL database
5. PostgreSQL returns the matching records to Prisma
6. Express sends the results back to React as a JSON response
7. React renders the results on the screen

2.3 Project Folder Structure

```
CS275/
├── client/                React frontend (Vite)
│   ├── src/
│   │   ├── App.jsx
│   │   └── main.jsx
├── routes/               Express API route handlers
│   ├── restaurants.js
│   └── favorites.js
├── prisma/               Database schema and migrations
│   ├── schema.prisma
│   ├── seed.js
│   └── migrations/
├── server.js             Express server entry point
├── prisma.config.ts      Prisma configuration
├── package.json
└── .env                  Database connection string
```


3. Database Design

The module uses PostgreSQL as the database with Prisma as the ORM. The schema defines three models: Restaurant, User, and Favorite. The User and Restaurant tables are currently placeholder implementations — they will be replaced with the canonical tables owned by the authentication and listings modules respectively once those schemas are finalized.

3.1 Tables

3.1.1 Restaurant

Stores restaurant listing data. Owned by the listings module; a placeholder version is used locally for development.

Column	Type	Description
id	Int (PK)	Auto-incremented unique identifier
name	String	Restaurant name
cuisine	String	Cuisine type (e.g. Italian, Japanese)
price	String	Price range using \$ / \$\$ / \$\$\$ / \$\$\$\$ notation
rating	Float	Aggregate star rating, defaults to 0
location	String	City or neighbourhood (e.g. Vancouver, BC)
description	String?	Optional description of the restaurant
createdAt	DateTime	Timestamp of when the listing was added

3.1.2 User

Stores user account data. Owned by the authentication module; a placeholder version is used locally for development.

Column	Type	Description
id	Int (PK)	Auto-incremented unique identifier
name	String	User's full name
email	String	Unique email address used for login
password	String	Hashed password
createdAt	DateTime	Timestamp of when the account was created

3.1.3 Favorite

Owned exclusively by the Discovery module. Acts as a join table connecting a User to a Restaurant they have saved.

Column	Type	Description
id	Int (PK)	Auto-incremented unique identifier

userId	Int (FK)	References User.id — the user who saved the restaurant
restaurantId	Int (FK)	References Restaurant.id — the restaurant that was saved
createdAt	DateTime	Timestamp of when the restaurant was saved

3.2 Relationships

- One User can have zero or many Favorites (one-to-many)
- One Restaurant can be saved as zero or many Favorites (one-to-many)
- Favorite is a join table — it connects User and Restaurant in a many-to-many relationship

Notation: User ||--o{ Favorite and Restaurant ||--o{ Favorite

3.3 Shared Tables & Dependencies

Table	Owner	Our Dependency
Favorite	This module	Fully owned — no dependency
Restaurant	Listings module	We query Restaurant for search/filter; Favorite.restaurantId references Restaurant.id
User	Auth module	Favorite.userId references User.id; favorites cannot be saved without a valid user

4. API Design

The backend exposes a RESTful API built with Express. All responses are in JSON format. The server runs on port 3000 during development (<http://localhost:3000>).

4.1 Restaurant Routes

Method	Endpoint	Description
GET	/restaurants	Returns all restaurants. Supports query params: search, cuisine, price, rating, sort
GET	/restaurants/:id	Returns a single restaurant by its ID. Returns 404 if not found.
POST	/restaurants	Creates a new restaurant. Expects name, cuisine, price, rating, location, description in the request body.

Search & Filter Query Parameters

Parameter	Description
search	Searches restaurant name, cuisine, and description (case-insensitive, partial match)
cuisine	Filters by cuisine type (e.g. ?cuisine=Italian)
price	Filters by price range (e.g. ?price=\$\$)
rating	Filters by minimum rating (e.g. ?rating=4 returns restaurants rated 4.0 and above)
sort	Sorts results. Options: newest (by createdAt descending). Default: rating descending.

4.2 Favorites Routes

Method	Endpoint	Description
GET	/favorites/:userId	Returns all saved restaurants for a given user, including full restaurant details.
POST	/favorites	Saves a restaurant to favorites. Expects userId and restaurantId in the request body.
DELETE	/favorites/:id	Removes a favorite entry by its ID.

4.3 HTTP Status Codes Used

Code	Meaning
200 OK	Request succeeded

201 Created	A new resource was successfully created (restaurant or favorite)
404 Not Found	The requested resource does not exist
500 Server Error	An unexpected error occurred on the server

5. UI/UX Design

The frontend is built with React using Vite as the build tool. The interface is a Single Page Application (SPA) — meaning the page does not reload when the user navigates or applies filters. All UI updates happen dynamically by React re-rendering components when data changes.

5.1 Key Pages

5.1.1 Main Discovery Page

The primary page users land on. It contains all core discovery features in one view without requiring navigation away from the results list (NFR-DS-003). The layout consists of:

- A search bar at the top for keyword search
- A filter panel on the left sidebar with cuisine, price, rating, and location filters
- A sort control (by rating or newest)
- A Clear All Filters button (FR-DS-SF-019)
- A results grid showing restaurant cards
- Each card displays the restaurant name, cuisine, price, rating, a bookmark icon, and a share button

5.1.2 Restaurant Detail Page

Accessed when a user clicks on a restaurant card. Shows full restaurant details including name, cuisine, location, price, rating, and description. Includes a bookmark/heart icon to save to favorites and a share button to copy the URL.

5.1.3 Favorites Page

Only accessible to registered users. Displays a list of all restaurants the user has saved. If no favorites exist, displays an empty state message. Each item includes an option to unsave the restaurant.

5.2 UI Component Design Decisions

Component	Design Decision
Search Bar	Always visible at the top of the page. Empty state shows all listings (FR-DS-SF-004). No results state shows a friendly message with suggestion to broaden search (FR-DS-SF-005).
Cuisine Filter	Rendered as a multi-select list derived from the database. Allows selecting multiple cuisines simultaneously (FR-DS-SF-007).
Price Filter	Uses \$ / \$\$ / \$\$\$ / \$\$\$\$ notation. Multi-select enabled (FR-DS-SF-010, FR-DS-SF-011).
Rating Filter	Minimum star rating selector (3+, 4+, 5 stars). Each listing card shows the current aggregate rating (FR-DS-SF-012, FR-DS-SF-013).
Bookmark Icon	Filled icon = saved, outline icon = not saved. Guest users are prompted to log in when clicking (FR-DS-UI-028, FR-DS-UI-029, FR-DS-UI-031).

Share Button	Copies the restaurant URL to clipboard and displays a brief 'Link copied!' confirmation toast (FR-DS-UI-033, FR-DS-UI-034, FR-DS-UI-035). Available to both guest and registered users (FR-DS-UI-036).
Clear All Filters	A visible button that resets all active filters simultaneously and reverts to the default sort order (FR-DS-SF-019).

5.3 Responsive Design

The interface is designed to be usable on desktop, tablet, and mobile screen sizes (NFR-DS-004). On smaller screens the filter panel collapses into a modal or drawer. Restaurant cards stack vertically on mobile. The search bar and sort controls remain accessible at all screen sizes.

5.4 Error Handling

The system provides clear error messages for common failure cases (NFR-DS-009):

- Empty search with no results: 'No restaurants found. Try broadening your search.'
- Network or server error: 'Something went wrong. Please try again.'
- Guest user attempting to save: 'Please log in to save restaurants.'
- Clipboard unavailable during share: displays the URL directly for the user to copy manually

6. Work Division

Team Member	Responsibilities
Edrien Borigas	Database schema, Express server, Prisma setup, API routes (restaurants & favorites), database seeding, GitHub repository management
Rajvir Kaur	Search & Results — FR-DS-SF-001 to 005 (keyword search input, search by name/keyword, empty state, no results state)
Vitumbiko Ngwira	Filters — FR-DS-SF-006, 007, 010, 012, 019 (cuisine filter, price filter, rating filter, clear all filters)
Gursharn S. Sangha	Sort & Display — FR-DS-SF-013, 014, 016, 017 (rating display, sort by rating, sort by newest, default sort order)
Ramanjeet Kaur	Favorites & Share — FR-DS-UI-028 to 036 (bookmark control, auth gate, favorites persistence, visual saved state, favorites list, share button, copy link, copy confirmation, guest sharing)

7. Non-Functional Requirements

NFR ID	Requirement
NFR-DS-001	Search results load and render within 2 seconds under normal network conditions
NFR-DS-003	Search bar, filters, and sort controls accessible from a single page without navigation
NFR-DS-004	Interface is responsive and usable on desktop, tablet, and mobile
NFR-DS-006	Interactive controls have appropriate ARIA labels for screen reader accessibility
NFR-DS-007	Search and filter backend handles up to 10,000 restaurant listings without degraded response times
NFR-DS-008	Application functions correctly on latest versions of Chrome, Firefox, and Edge
NFR-DS-009	System provides clear, user-friendly error messages when invalid input or system errors occur